Instructor Handout 2: Number Systems, Representation, and Arithmetic Hardware

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

August 8, 2026

Instructor Copy — Not for Student Distribution

2.1 Before You Teach This Lecture

- **Patterson & Hennessy, *Computer Organization and Design*, ARM Edition** — Sec. 3.2 "Addition and Subtraction," p.188 (two's complement, overflow); Sec. 3.3 "Multiplication," p.191; Sec. 3.4 "Division," p.197. Page-cited — index built 2026-08-07. Use Sec. 3.2 for the precise overflow condition and the "$A - B = A + (-B)$" free-subtraction point.
- **Hamacher, *Computer Organization*, Ch. 2** and **Mano, *Computer System Architecture*, Ch. 1** — *not page-indexed.* Cite at chapter level for number representation and complement arithmetic.
- **Do the thread arithmetic before class**: $+7+(-3)$ in 8-bit two's complement ($0000\,0111 + 1111\,1101 = 1\,0000\,0100$, discard carry, result $0000\,0100 = +4$), then the same in 4-bit ($0111 + 1101 = 0100$, carry-out 1). Have both ready so the board numbers are right the first time.
- **Plan the two's-complement range example**: 8-bit -128 to $+127$; the overflow cases $+100 + 50$ and $+127 + 1$ vs. the correct $-1 + -1$.

2.2 Number Systems

Open by asking how to write the decimal 13 with only two symbols (or eight, or sixteen) — the point being that each is the *same* value in a different alphabet, and that hardware stores *bits* while the base is just notation. Then give the positional definition.

Definition (Positional Number System). *In base r, a number with digits $d_{n-1} \ldots d_1 d_0$ has value $d_{n-1} r^{n-1} + \cdots + d_1 r^1 + d_0 r^0$, with the base written as a subscript.*

Introduce the common bases (binary $r = 2$, octal $r = 8$, decimal $r = 10$, hex $r = 16$) and the two conversion facts that matter: repeated division converts decimal to any base, and hex/octal are just binary in groups (1 hex digit = 4 bits, 1 octal digit = 3 bits).

Example 2.2.1 (Decimal to binary, and back)**.** $53_{10} = 110101_2$ *by repeated division by 2 (remainders read bottom-up); check by expanding* $32+16+4+1 = 53$*. And* $1101\,0101_2 = D5_{16}$ *by grouping — no arithmetic.*

How to present it: do the repeated division on the board as a *column of divisions* (dividend, quotient, remainder), not a jumble — students need to see where "bottom-up" comes from. Then do the check expansion *afterwards*, so the conversion is verified rather than asserted.

> **Teaching note:** The single most common student error here is reading the remainders *top-down* instead of bottom-up. Make the mistake explicit: "if I read these remainders top-down I get the reverse of the right answer — why?" Because the first division by 2 finds the *least* significant bit.

Close the section with the bit-counting Socratic question: n distinct values need $\lceil \log_2 n \rceil$ bits; k bits give 0 to $2^k - 1$ unsigned. State the 8-bit/16-bit ceilings — they are the setup for overflow later.

2.3 Signed Representation

Set up the problem: we have n bits and must encode a sign. Present the three candidate schemes as a design discussion, not a list — ask which one would make *addition* easiest.

Definition (Two's Complement). $-x \equiv 2^n - x \pmod{2^n}$*, equivalently "flip all bits and add 1." Range* $-2^{n-1} \le x \le 2^{n-1} - 1$*; sign bit = MSB;* $x + (-x) = 0$ *in the same width with the carry-out discarded.*

How to present it: build two's complement from the requirement that $x + (-x)$ must be 0 in n bits. Write 0000 0111 (+7) on the board and ask: "what bits must I add to this to get all zeros, with the carry-out thrown away?" The class can derive "flip everything and add 1" as the answer — which makes the definition feel discovered rather than imposed. (P&H, Sec. 3.2, p.188; Hamacher Ch. 2) [2, 1]

> **Teaching note:** Students memorize "flip and add 1" without understanding why the range is asymmetric (-2^{n-1} to $2^{n-1} - 1$) or why the MSB is the sign. Spend the time on $x + (-x) = 0$: it explains the rule, the range, and the "discard the carry" convention all at once.

Example 2.3.1 (+7 and −3 in 8-bit two's complement)**.** $+7 = 0000\,0111$*; −3: flip* $0000\,0011 \to 1111\,1100$*, add 1* $\Rightarrow 1111\,1101$*. Adding:* $0000\,0111 + 1111\,1101 = 1\,0000\,0100$*; discard the carry-out* $\Rightarrow 0000\,0100 = +4$*. The thread's first full example: negatives are ordinary bits, and addition needs no special subtraction hardware.*

A question worth asking live: "negate 1111 1011 (−5) and negate again — what do you get, and what property does that show?" (+5; two's complement is an involution, $-(-x) = x$.)

2.4 Overflow and Fixed-Point

Open with the definition and immediately hit the key subtlety: **overflow** ≠ carry-out. The thread's own example ($7 + (-3)$) already produced a carry-out with a correct result, so the class has seen the counterexample.

Definition (Overflow). *Occurs when the true result of an arithmetic operation cannot be represented in the format's range. For two's complement, overflow happens exactly when the two operands share a sign bit and the result's sign bit differs; carry-out alone is not the test.*

Example 2.4.1 (Overflow vs. carry in 8-bit two's complement)**.** *Range* -128 *to* $+127$*.* $+100+50 = 1001\,0110$*, which reads as* -106 *— wrong sign, overflow.* $+127+1 = 1000\,0000 = -128$ *— wrong sign, overflow.* $-1+-1 = 1\,1111\,1110$*; discard carry* $\Rightarrow 1111\,1110 = -2$ *— correct. Carry-out is present in both the overflow* and *the correct case, so it cannot be the test.*

> **Teaching note:** This is the lecture's hardest conceptual point, and it rewards a full example. The wrong-same-sign test is the version that survives: "did adding two positives produce a negative, or two negatives a positive?" Write the rule as a condition the class can apply mechanically, and test it on all three examples.

Then fixed point. The definition is short — an agreed position for the binary point — and the worked example makes it concrete: the *same* integer ALU adds fixed-point values; only the interpretation changes.

Example 2.4.2 (A fixed-point addition)**.** *Add* $3.5 + 0.75$ *in Q8.8:* $3.5 = 0000\,0011.1000\,0000$ *and* $0.75 = 0000\,0000.1100\,0000$*. Adding bitwise gives* $0000\,0100.0100\,0000 = 4.25$*. The integer adder did all the work; the programmer just agreed where the point is.*

Close with the Socratic question that motivates Week 3: 0.1 is a **repeating** binary fraction, so fixed point truncates and accumulates error — the limitation floating point manages (but does not eliminate).

2.5 Adder Hardware

Motivate with the full adder: the smallest circuit that adds a single bit with carry-in. Define the half and full adders, then the chain.

Definition (Half Adder and Full Adder).

$$HA\text{:}\ S = A \oplus B,\ C = A \cdot B; \qquad FA\text{:}\ S = A \oplus B \oplus C_{in},\ C_{out} = A \cdot B + C_{in} \cdot (A \oplus B).$$

Walk the full-adder truth table (Table 1) once, pointing out the counting pattern — sum is the parity of the inputs, carry is "two or more 1s."

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 1: Full adder truth table.

Then the ripple-carry adder (Figure 2.1).

How to present it: draw FA0 with its $C_{in} = 0$, then FA1, and *ask the class* where FA1's carry-in comes from before drawing the connecting wire. The "ripple" naming then writes itself — each carry propagates to the next stage.

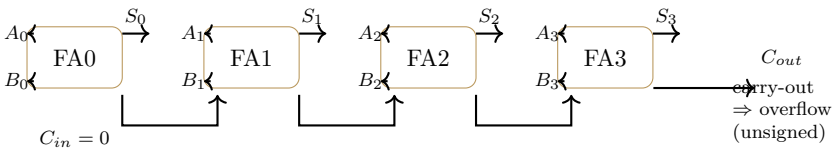

Figure 2.1: The ripple-carry adder: full adders chained by their carry signals.

> **Teaching note:** Students will ask why the carry has to ripple through all stages at all (delay). Answer briefly: it does, and that is exactly why carry-lookahead exists — but lookahead is the competitive-exam question (Q6 in the practice set) and a Week 11+ refinement, not today's material. Note it and move on.

Close with the thread and the free-subtraction insight.

Example 2.5.1 (Adding $7 + (-3)$ in hardware)**.** $+7 = 0111$, $-3 = 1101$ *(4-bit). Ripple-carry with* $C_{in} = 0$*: bit 0:* $1+1 = 0$ *c 1; bit 1:* $1+0+1 = 0$ *c 1; bit 2:* $1+1+1 = 1$ *c 1; bit 3:* $0+1+1 = 0$ *c 1. Result* $0100 = +4$*, carry-out 1 (discarded). One adder, same circuit, signed or unsigned — interpretation is the programmer's choice.*

A question worth asking live: "to compute $A - B$ with only an adder, what one extra step do you need?" $(A + (-B))$, i.e. a MUX choosing B or $\bar{B}$ plus a forced carry-in of 1 — subtraction is **free**, P&H, Sec. 3.2, p.188) [2].

2.6 Synthesis and Summary

Close the thread: $+7 + (-3)$ used every tool — two's complement representation, addition without subtraction hardware, the ripple-carry adder, the discarded carry-out, and the sign check for overflow. The result $0100 = +4$ is the same number a human computes, but now it is the output of a specific circuit.

Set up Week 3: fixed point handles fractions but with fixed range/precision; real numbers span 10^{-300} to 10^{300}; Week 3 builds IEEE 754 floating point — sign, exponent, mantissa — trading a little precision for a huge range.

Summary — quick reference: *Number systems:* value $= \sum d_i r^i$; convert by repeated division; hex/octal group bits. *Two's complement:* $-2^{n-1} \le x \le 2^{n-1} - 1$; negate by flip + 1; addition needs no special subtraction hardware. *Overflow:* carry-out $\neq$ overflow; wrong result sign is the test. *Fixed point:* an agreed binary point; simple but limited range and precision. *Adder:* HA/FA equations; ripple-carry chain; $A - B = A + (-B)$.

References

[1] V. Carl Hamacher, Zvonko G. Vranesic, and Safwat G. Zaky. *Computer Organization*. McGraw-Hill, 5th edition, 2002.

[2] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, arm edition (5th) edition, 2017.